Database Design

SmartSociety — Web-Based Housing Society Management System

Engine MySQL / MariaDB, InnoDB

Schema file `Backend/database/smart_society_schema.sql`

Date August 2026

1 Entity-Relationship Diagram

21 tables across six domains: identity, occupancy, billing, complaints, gate/security, and community. `flats` and `users` are the two hubs almost everything else hangs off. Shown as two views — the core operational spine, then the community/support tables — rather than one dense diagram, so each stays readable.

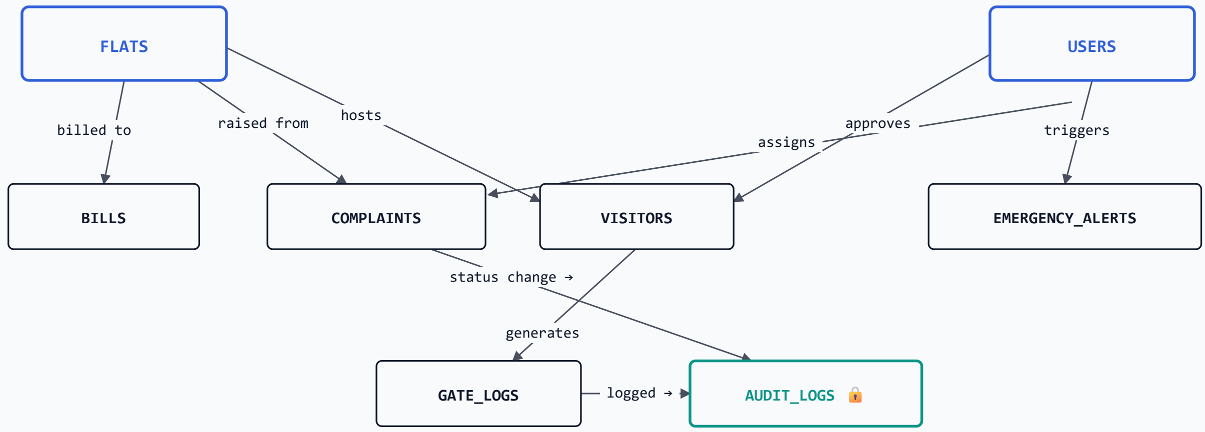

Fig. 1 — the core operational spine. FLATS and USERS are the two hubs; AUDIT_LOGS has no outgoing edges — it only receives, consistent with the BEFORE UPDATE/DELETE triggers that make it append-only (§3).

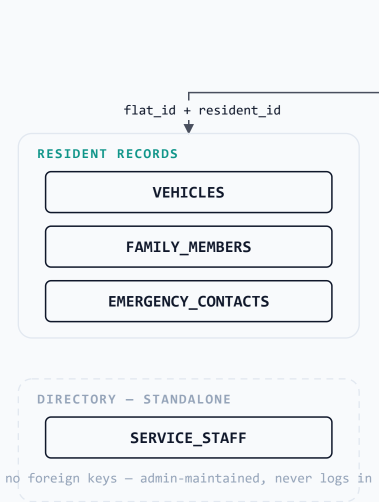

Fig. 2 — community and support tables, grouped by relationship pattern rather than wired individually: most hang off both FLATS and USERS the same way (§2's "resident + flat" pairing); AMENITIES and POLLS each own a small sub-hierarchy; SERVICE_STAFF has no foreign keys at all.

2 Data Dictionary

Every table's primary key is the literal column `_id` (auto-increment integer). Foreign keys are plain integer columns named after the referenced entity (e.g. `flat_id`, `resident_id`). ■ primary key, ■ foreign key.

Identity & Access

TABLE	KEY COLUMNS	NOTES
`users`	_id, username, email (unique), password (bcrypt), role, flat_id	role ∈ {Admin, Resident, Guard}; flat_id null for Admin/Guard
`login_otps`	_id, user_id, code, expires_at, consumed	5-min TTL, single-use, sole gate to JWT issuance
`password_resets`	_id, user_id, token (unique), expires_at, consumed	30-min TTL, crypto-random token

Occupancy

TABLE	KEY COLUMNS	NOTES
`flats`	_id, block_name + flat_number (unique pair), occupancy_type, is_occupied	occupancy_type ∈ {Owner, Tenant}
`family_members`	_id, resident_id, flat_id, relationship	relationship includes "Tenant" as a first-class value
`vehicles`	_id, resident_id, flat_id, vehicle_number (unique)	
`emergency_contacts`	_id, resident_id, flat_id, is_primary	

Billing

TABLE	KEY COLUMNS	NOTES
`bills`	_id, flat_id, billing_month, total_due, payment_status	money fields are FLOAT — MariaDB returns DECIMAL as a string via mysql2

Complaints

TABLE	KEY COLUMNS	NOTES
`complaints`	_id, resident_id, flat_id, assigned_to, priority, status, sla_due_date	sla_due_date computed at assignment from priority (Urgent 4h / High 24h / Medium 48h / Low 120h)

Gate & Security

TABLE	KEY COLUMNS	NOTES
`visitors`	_id, flat_id, approved_by, gate_pass_code, qr_code, status	status ∈ {Pre-Approved, Entered, Exited}
`gate_logs`	_id, visitor_id, flat_id, logged_by, entry_time, exit_time	every entry is also written to audit_logs
`security_alerts`	_id, visitor_id, alert_type, alert_status	Overstay rows created by the 5-minute background monitor job
`emergency_alerts`	_id, triggered_by, flat_id (nullable), status	triggered_by_name / _role denormalized at write time; broadcast via Socket.io

Community

TABLE	KEY COLUMNS	NOTES
`amenities`	_id, name, capacity, availability	
`amenity_bookings`	_id, amenity_id, resident_id, flat_id, booking_status	
`notices`	_id, created_by, category, urgent	
`polls`	_id, created_by, poll_status, total_votes	
`poll_options`	_id, poll_id, vote_count	ON DELETE CASCADE from polls
`votes`	_id, poll_id + resident_id (unique pair)	unique constraint enforces one vote per resident per poll
`service_staff`	_id, name, service_type, is_active	no auth columns — directory entry, never logs in

Audit

TABLE	KEY COLUMNS	NOTES
`audit_logs`	_id, entity_type, entity_id, performed_by, before_value, after_value	entity_type ∈ {GateEntry, Complaint, Bill}; no `updatedAt` column — rows are never updated

3 Immutability Enforcement

DB-LEVEL GUARANTEE

`audit_logs` carries two `BEFORE UPDATE` / `BEFORE DELETE` triggers that raise `SIGNAL SQLSTATE '45000'` on any attempt to modify or remove a row. This holds even against a direct database connection that bypasses the application entirely — the guarantee is enforced by MySQL itself, not by application-layer discipline. The triggers are installed idempotently on every server boot, so the guarantee survives a fresh database import from the schema file.